Day 4 — Lab 4.3: Event-Driven Data Pipeline using ADF and Azure Functions

1. Business scenario

Your organization receives order files in ADLS Gen2 from external business systems.

Currently, the operations team manually starts the data pipeline whenever a new file arrives.

The requirement is to automate this process:

- Detect when a new CSV file arrives in ADLS Gen2.
- Automatically trigger an Azure Data Factory pipeline.
- Invoke a C# Azure Function from the pipeline.
- Validate the incoming file notification.
- Log the file details and return an acknowledgement.

Participants will implement the integration and verify the complete event-driven execution.

2. Expected architecture

ADLS Gen2

New CSV file arrives in raw/lab43

Storage Event — Event Grid

Azure Data Factory

Storage Event Trigger starts pipeline

Azure Function Activity — HTTP POST

Azure Function — C# .NET 10

Validate notification, log file details and return acknowledgement

Important distinction: Event Grid triggers ADF. ADF then calls the Azure Function. The Function is not directly subscribed to the storage event in this lab.

3. Resources required

Reuse the Day 4 resources from Labs 4.1 and 4.2.

Resource	Example name
Storage Account	`stadfuser01d4`
Data Factory	`adf-user01-d4`
Function App	`fn-user01-adf-d4`
Function project	`day04-event-notifier`

The Function App is the additional resource for this lab. Each participant creates their own.

Use a supported hosting plan for .NET 10 isolated worker. For Linux, use Flex Consumption rather than Linux Consumption.

4. Prerequisites

- Lab 4.1 and Lab 4.2 environment is available.
- ADLS Gen2 and Data Factory are working.
- Azure Functions Core Tools v4.
- .NET 10 SDK.
- VS Code with C# tooling.
- Permission to create the Function App and Event Grid subscription.
- Permission to register the Event Grid resource provider if it is not already registered.

5. Create the ADLS Gen2 folder

We will use the existing Day 4 Storage Account.

Navigate to:

Azure Portal → Storage Account → Containers → raw

Create a folder:

```
None
raw/
└── lab43/
```

This folder will receive the event-triggering CSV file.

Do not upload the test file yet. We will upload it after activating the trigger.

6. Create an independent C# Function project

Just like our earlier days, Lab 4.3 gets its own independent project.

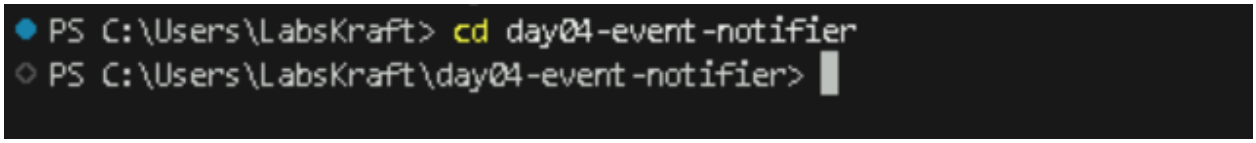
Open VS Code and navigate to your Day 4 working directory.

Open Terminal and execute:

```
func init day04-event-notifier --worker-runtime dotnet-isolated --target-framework net10.0
```

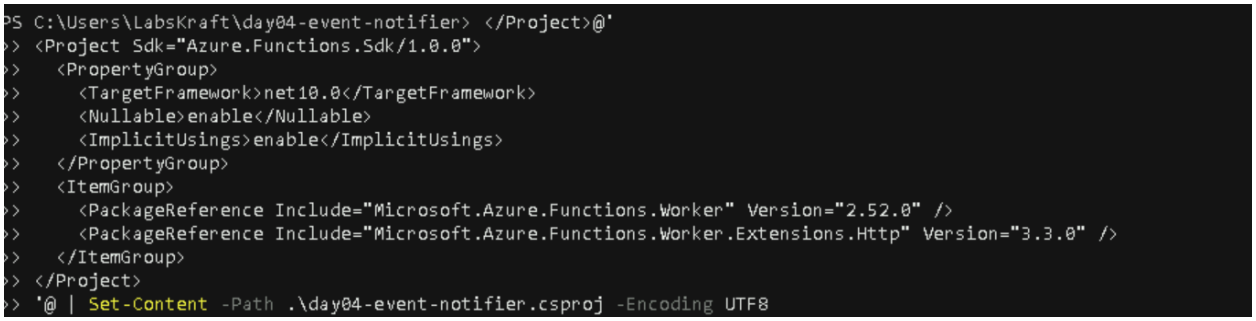
Navigate into the project:

```
cd day04-event-notifier
```



Create a new C# file:

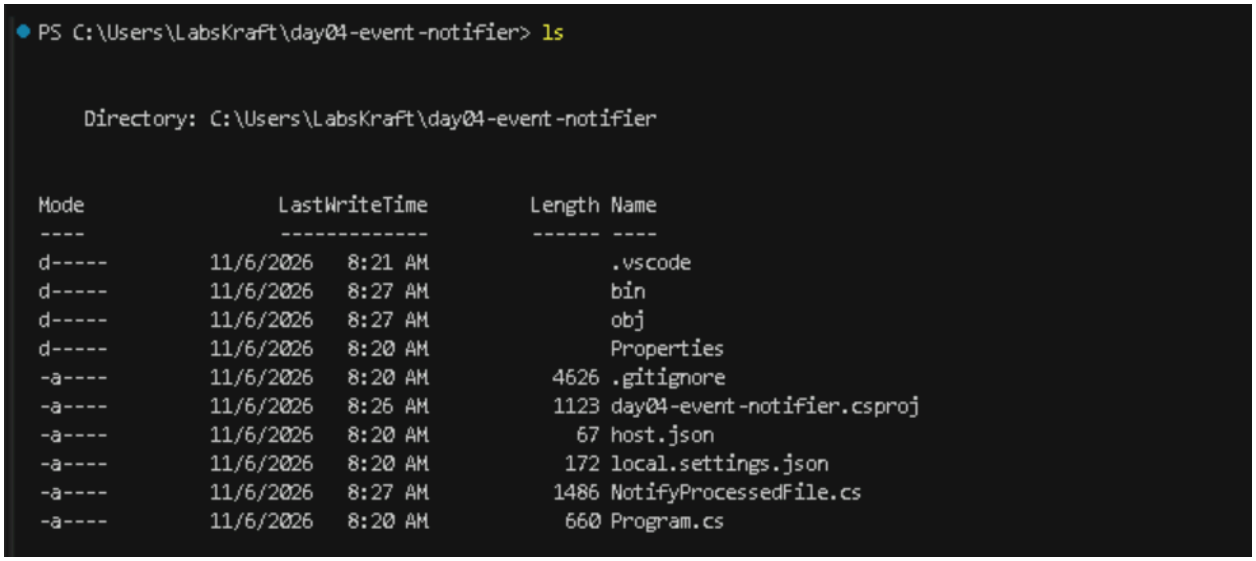
```
NotifyProcessedFile.cs
```



Your project should look like:

```
day04-event-notifier/
|
├── Program.cs
```

```
├── day04-event-notifier.csproj
├── host.json
├── local.settings.json
└── NotifyProcessedFile.cs
```



This project is separate from the Day 2 and Day 3 projects.

7. Implement the C# Azure Function

Open `NotifyProcessedFile.cs` and add:

None

```
using System.Net;
using Microsoft.Azure.Functions.Worker;
using Microsoft.Azure.Functions.Worker.Http;
using Microsoft.Extensions.Logging;

namespace Day04EventNotifier;
```

```
public class NotifyProcessedFile
{
    private readonly ILogger<NotifyProcessedFile>
_logger;

    public NotifyProcessedFile(
        ILogger<NotifyProcessedFile> logger)
    {
        _logger = logger;
    }

    [Function("NotifyProcessedFile")]
    public async Task<HttpResponseData> Run(
        [HttpTrigger(
            AuthorizationLevel.Function,
            "post",
            Route = "NotifyProcessedFile")]
        HttpRequestData request)
    {
        var payload =
            await
request.ReadFromJsonAsync<FileNotification>();

        if (payload == null ||

string.IsNullOrWhiteSpace(payload.FileName))
        {
            var badResponse =
```

```
request.CreateResponse(HttpStatusCode.BadRequest);

            await badResponse.WriteStringAsync(
                "FileName is required.");

            return badResponse;
        }

        _logger.LogInformation(
            "Received file notification: {FileName}",
            payload.FileName);

        var response =

request.CreateResponse(HttpStatusCode.OK);

        await response.WriteAsJsonAsync(new
        {
            status = "accepted",
            fileName = payload.FileName,
            eventType = payload.EventType,
            receivedAtUtc = DateTimeOffset.UtcNow
        });

        return response;
    }
}
```

```
public class FileNotification
{
    public string? FileName { get; set; }
    public string? EventType { get; set; }
}
```

What this Function does:

- Receives an HTTP POST request from ADF.
- Reads the incoming JSON.
- Checks whether `fileName` is supplied.
- Logs the file name.
- Returns an acknowledgement with HTTP 200.
- Returns HTTP 400 if the file name is missing.

```
PS C:\Users\LabsKraft\day04-event-notifier> Set-Content -Path .\NotifyProcessedFile.cs -Value @"
>> using System.Net;
>> using Microsoft.Azure.Functions.Worker;
>> using Microsoft.Azure.Functions.Worker.Http;
>> using Microsoft.Extensions.Logging;
>>
>> namespace Day04EventNotifier;
>>
>> public class NotifyProcessedFile
>> {
>>     private readonly ILogger<NotifyProcessedFile> _logger;
>>
>>     public NotifyProcessedFile(ILogger<NotifyProcessedFile> logger)
>>     {
>>         _logger = logger;
>>     }
>>
>>     [Function("NotifyProcessedFile")]
>>     public async Task<HttpResponseData> Run(
>>         [HttpTrigger(AuthorizationLevel.Function, "post", Route = "NotifyProcessedFile")] HttpRequestData request)
>>     {
>>         var payload = await request.ReadFromJsonAsync<FileNotification>();
>>
>>         if (payload == null || string.IsNullOrWhiteSpace(payload.FileName))
>>         {
```

```
            await badResponse.WriteStringAsync("FileName is required.");
            return badResponse;
        }

        _logger.LogInformation("Received file notification: {FileName}", payload.FileName);

        var response = request.CreateResponse(HttpStatusCode.OK);
        await response.WriteAsJsonAsync(new
        {
            status = "accepted",
            fileName = payload.FileName,
            eventType = payload.EventType,
            receivedAtUtc = DateTimeOffset.UtcNow
        });

        return response;
    }
}

public class FileNotification
{
    public string? FileName { get; set; }
    public string? EventType { get; set; }
}
"@ -Encoding UTF8
```

This is our custom C# business logic. It does not yet read or transform the CSV file.

8. Verify the project dependencies

Open:

```
None
day04-event-notifier.csproj
```

For .NET 10 isolated worker, use the Azure Functions SDK and compatible worker packages. A Microsoft-documented baseline is:

```
None
<Project Sdk="Azure.Functions.Sdk/1.0.0">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <PackageReference
      Include="Microsoft.Azure.Functions.Worker"
      Version="2.52.0" />

    <PackageReference

Include="Microsoft.Azure.Functions.Worker.Extensions.
Http"
      Version="3.3.0" />
  </ItemGroup>

</Project>
```

```
PS C:\Users\LabsKraft\day04-event-notifier> Get-Content .\day04-event-notifier.csproj
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <AzureFunctionsVersion>v4</AzureFunctionsVersion>
    <OutputType>Exe</OutputType>
    <ImplicitUsings>enable</ImplicitUsings>
    <Nullable>enable</Nullable>
    <RootNamespace>day04_event_notifier</RootNamespace>
  </PropertyGroup>

  <ItemGroup>
    <FrameworkReference Include="Microsoft.AspNetCore.App" />
    <PackageReference Include="Microsoft.Azure.Functions.Worker" Version="2.52.0" />
    <PackageReference Include="Microsoft.Azure.Functions.Worker.OpenTelemetry" Version="1.2.0" />
    <PackageReference Include="OpenTelemetry.Extensions.Hosting" Version="1.15.3" />
    <PackageReference Include="Azure.Monitor.OpenTelemetry.Exporter" Version="1.7.0" />
    <PackageReference Include="Microsoft.Azure.Functions.Worker.Extensions.Http.AspNetCore" Version="2.1.1" />
    <PackageReference Include="Microsoft.Azure.Functions.Worker.Sdk" Version="2.0.7" />
  </ItemGroup>

</Project>
```

These are pinned baseline versions, not a claim that your local environment has already passed a build.

Run:

```
None
dotnet restore
```

Then:

```
None
dotnet build
```

Expected result
Build succeeded with zero errors.

```
PS C:\Users\LabsKraft\day04-event-notifier> dotnet clean

Build succeeded in 2.1s
PS C:\Users\LabsKraft\day04-event-notifier> dotnet build
Restore complete (1.0s)
  WorkerExtensions net8.0 succeeded (12.6s) → obj\Debug\net10.0\WorkerExtensions\bin\Release\net8.0\Microsoft.Azure.Functions.Worker.Extens
ions.dll
  day04-event-notifier net10.0 succeeded (18.7s) → bin\Debug\net10.0\day04-event-notifier.dll

Build succeeded in 40.1s
PS C:\Users\LabsKraft\day04-event-notifier>
```

The .NET 10 isolated-worker model is supported by Functions v4. For Linux hosting, use Flex Consumption rather than Linux Consumption.

Microsoft Learn+1

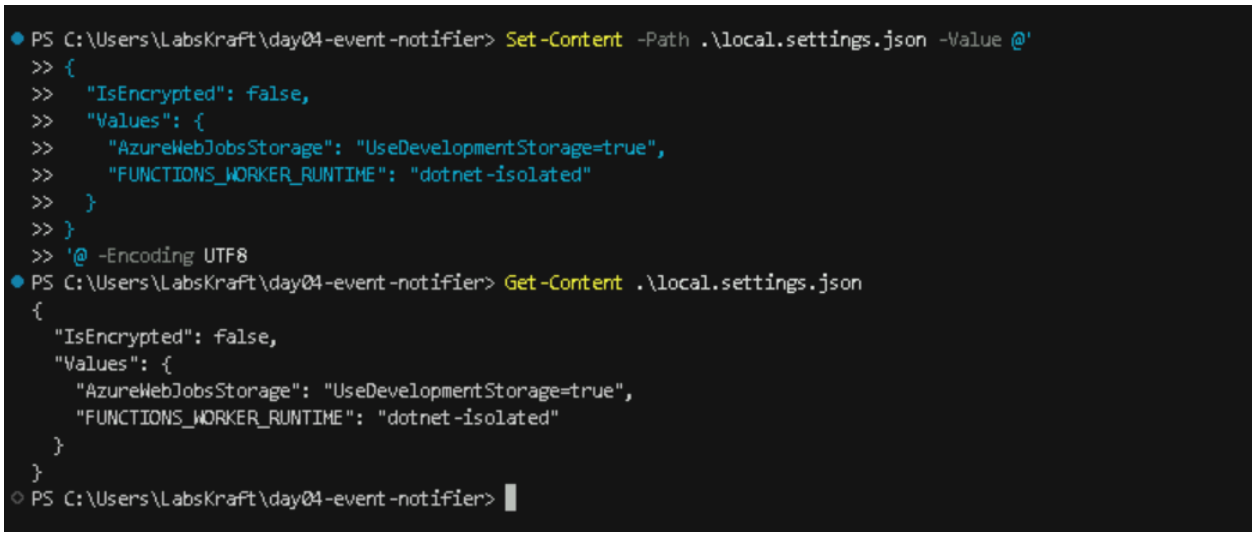
9. Test the Function locally

Before deploying to Azure, validate the C# Function locally.

For local execution, start Azurite so the Functions host has a local storage connection available.

In `local.settings.json`, configure:

```
None
{
  "IsEncrypted": false,
  "Values": {
    "AzureWebJobsStorage":
"UseDevelopmentStorage=true",
    "FUNCTIONS_WORKER_RUNTIME": "dotnet-isolated"
  }
}
```

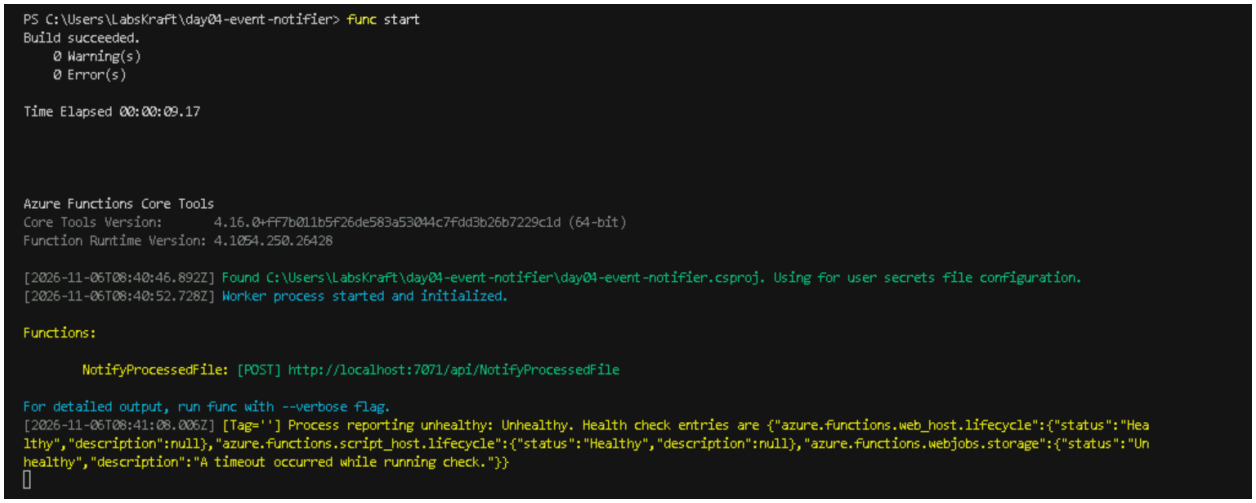


Start the Function:

```
None
func start
```

Expected function endpoint:

```
None
http://localhost:7071/api/NotifyProcessedFile
```



Open a second terminal and execute this PowerShell test:

```
None
$body = @{
    fileName = "event_orders.csv"
    eventType = "BlobCreated"
} | ConvertTo-Json

Invoke-RestMethod `
    -Uri
"http://localhost:7071/api/NotifyProcessedFile" `
    -Method Post `
    -ContentType "application/json" `
```

```
    -Body $body
```

Expected response:

```
None
{
  "status": "accepted",
  "fileName": "event_orders.csv",
  "eventType": "BlobCreated"
}
```

```
PS C:\Users\LabsKraft> $body = @{
>>     fileName = "event_orders.csv"
>>     eventType = "BlobCreated"
>> } | ConvertTo-Json
PS C:\Users\LabsKraft>
PS C:\Users\LabsKraft> Invoke-RestMethod `
>>     -Uri "http://localhost:7071/api/NotifyProcessedFile" `
>>     -Method Post `
>>     -ContentType "application/json" `
>>     -Body $body

status   fileName         eventType   receivedAtUtc
------   --------         ---------   -------------
accepted event_orders.csv BlobCreated 2026-11-06T08:42:14.1590206+00:00


PS C:\Users\LabsKraft>
```

Confirm the Function log also displays the file name.

```
[2026-11-06T08:42:13.790Z] Executing 'Functions.NotifyProcessedFile' (Reason='This function was programmatically called via the host APIs.', Id=a53aa988-530a-48bf-9042-b772ba4012fd)
[2026-11-06T08:42:14.167Z] Received file notification: event_orders.csv
[2026-11-06T08:42:14.473Z] Executed 'Functions.NotifyProcessedFile' (Succeeded, Id=a53aa988-530a-48bf-9042-b772ba4012fd, Duration=748ms)
[2026-11-06T08:42:27.953Z] [Tag=''] Process reporting unhealthy: Unhealthy. Health check entries are {"azure.functions.web_host.lifecycle":{"status":"Healthy","description":null},"azure.functions.script_host.lifecycle":{"status":"Healthy","description":null},"azure.functions.webjobs.storage":{"status":"Unhealthy","description":"Unable to access AzureWebJobsStorage","errorCode":null}}
[2026-11-06T08:42:57.954Z] [Tag=''] Process reporting unhealthy: Unhealthy. Health check entries are {"azure.functions.web_host.lifecycle":{"status":"Healthy","description":null},"azure.functions.script_host.lifecycle":{"status":"Healthy","description":null},"azure.functions.webjobs.storage":{"status":"Unhealthy","description":"Unable to access AzureWebJobsStorage","errorCode":null}}
```

Stop the local host after successful validation.

10. Create the Azure Function App

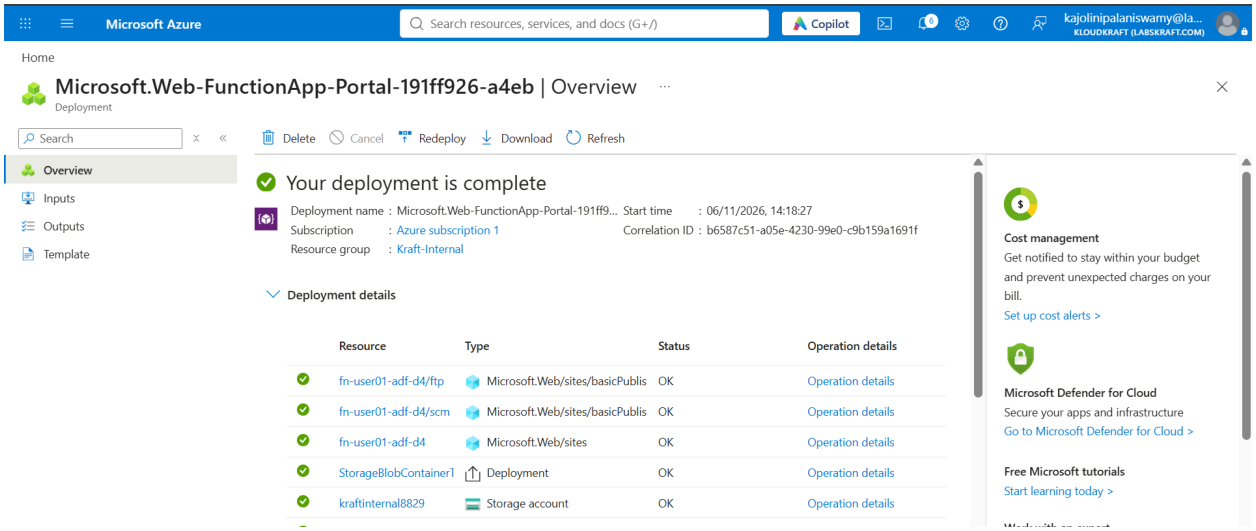
Navigate to:

Azure Portal → Create a resource → Function App

Configure:

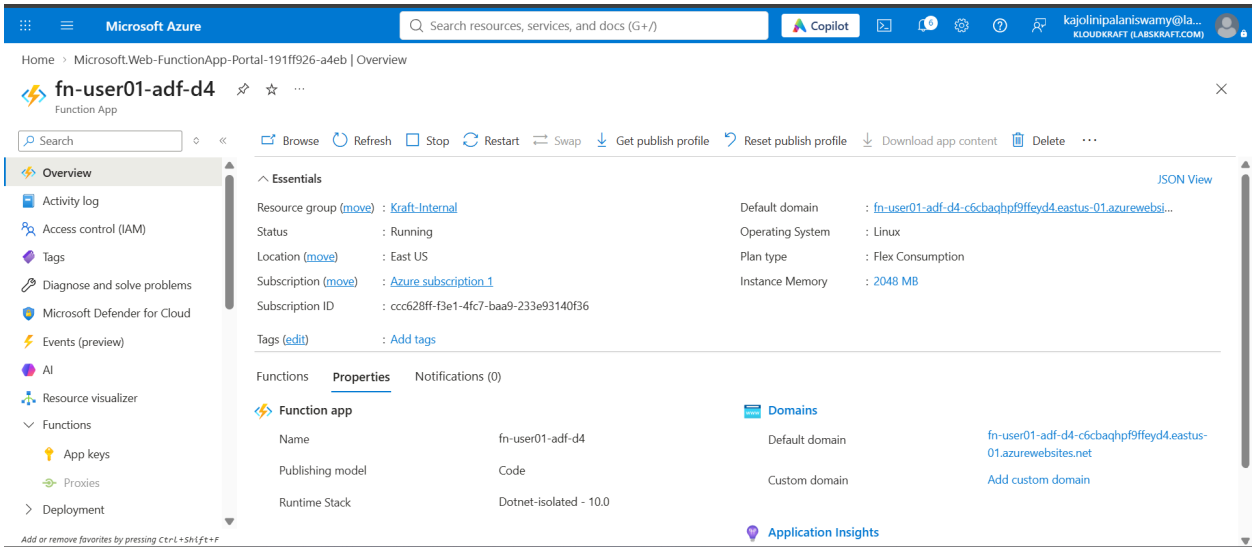
Setting	Value
Function App	`fn-user01-adf-d4`
Resource Group	Existing shared RG
Runtime stack	.NET
Version	.NET 10 isolated
Operating System	Linux
Hosting plan	Flex Consumption
Region	Approved training region

Complete the remaining required configuration and create the Function App.



After deployment:

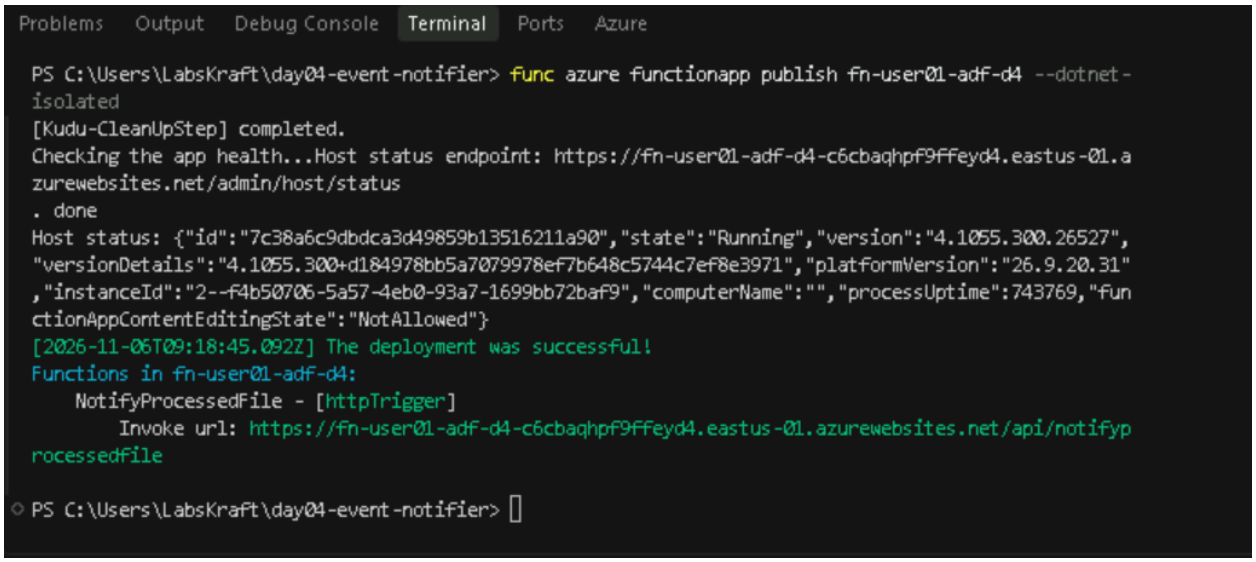
1. Open the Function App.
2. Navigate to Configuration or Environment variables.
3. Verify the runtime settings.
4. Confirm Functions runtime version is 4.
5. Confirm the Function App is running.



11. Deploy the C# Function

In VS Code:

1. Open the `day04-event-notifier` project.
2. Install or open the Azure Functions extension.
3. Sign in to Azure.
4. Select your subscription.
5. Select the Function App `fn-user01-adf-d4`.
6. Deploy the project.



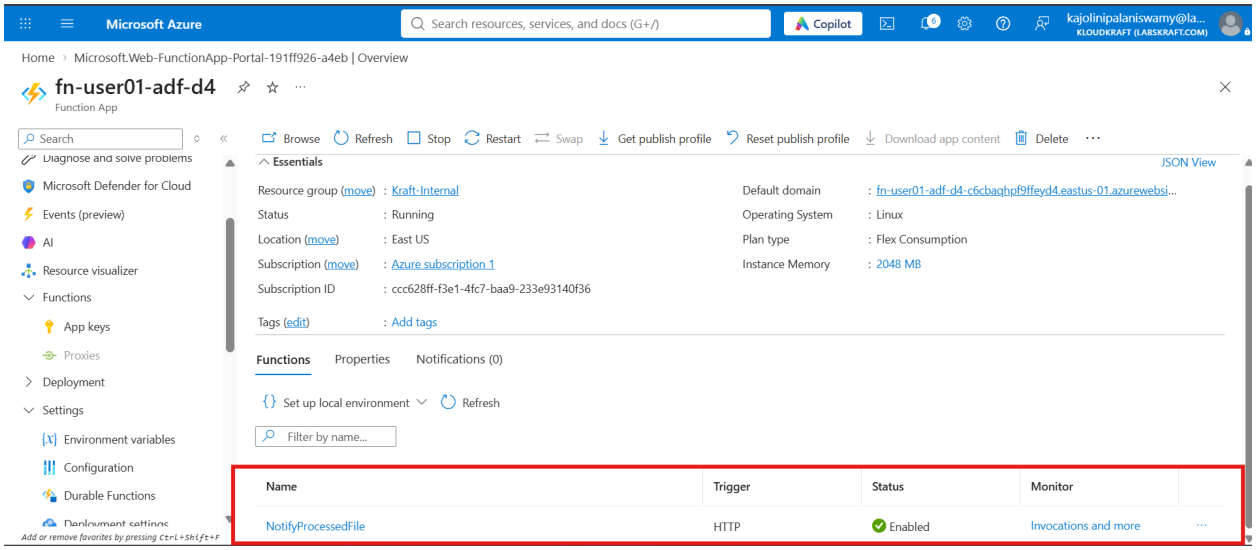
After deployment:

Navigate to:

Azure Portal → Function App → Functions

Verify that this function is listed:

```
None
NotifyProcessedFile
```

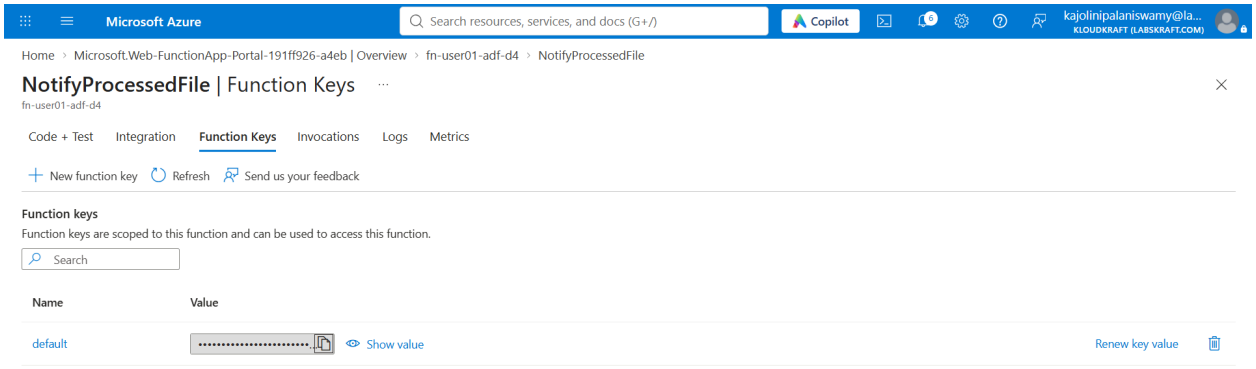


Open the function and confirm it is available.

Obtain the Function key

1. Open `NotifyProcessedFile`.
2. Navigate to **Function keys**.
3. Copy the default function key.

Keep the key secure. Do not place it in source code, screenshots or the participant guide.



Checkpoint 1
The C# Function is deployed and visible in Azure.

12. Create the Azure Function linked service in ADF

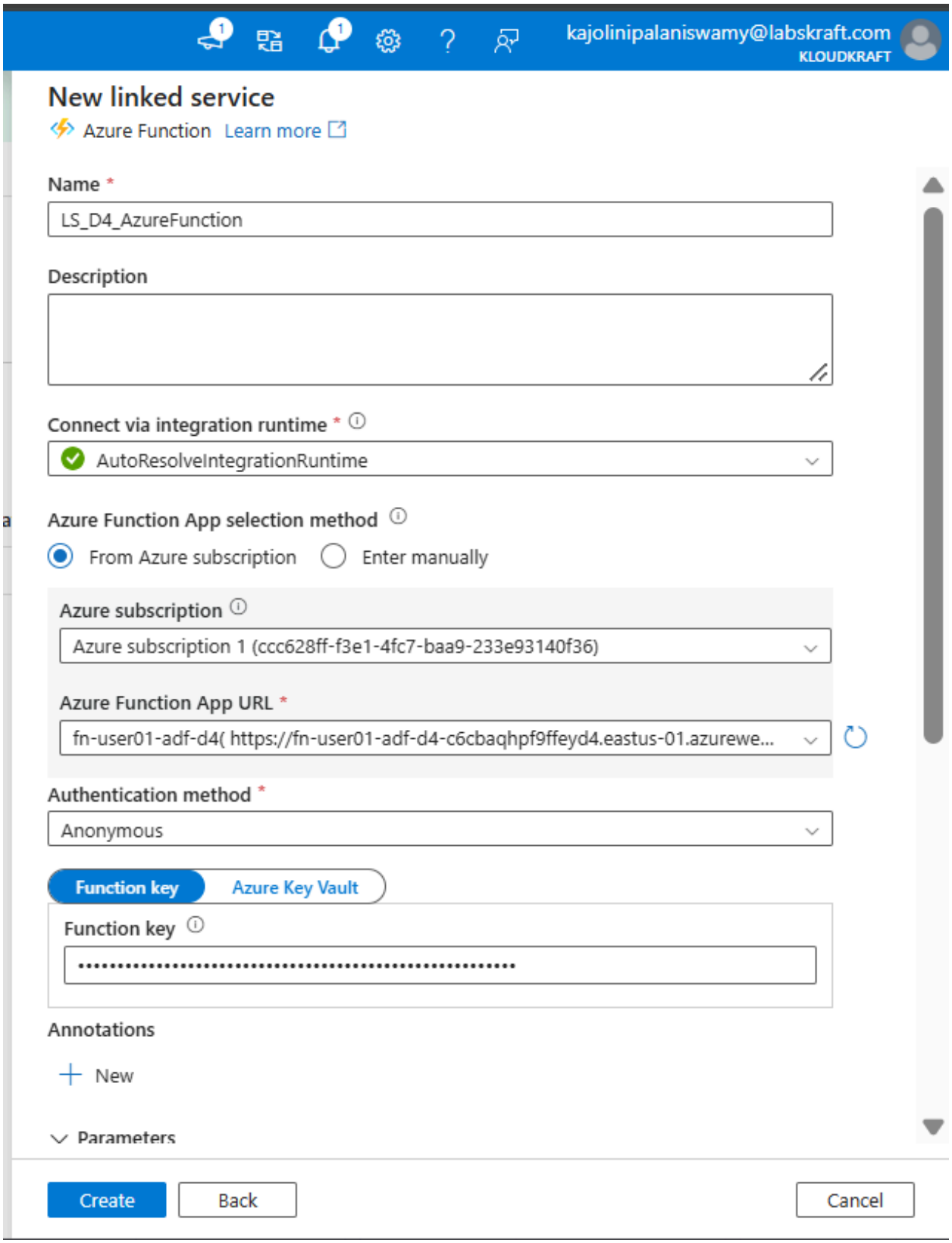
Open Azure Data Factory Studio → Manage → Linked services → + New.

1. Search for Azure Function.
2. Select Azure Function.
3. Select Continue.

Configure:

Setting	Value
Name	`LS_D4_AzureFunction`
Function App URL	Your Function App URL
Authentication	Function key
Function key	Your securely copied key
Integration Runtime	AutoResolveIntegrationRuntime

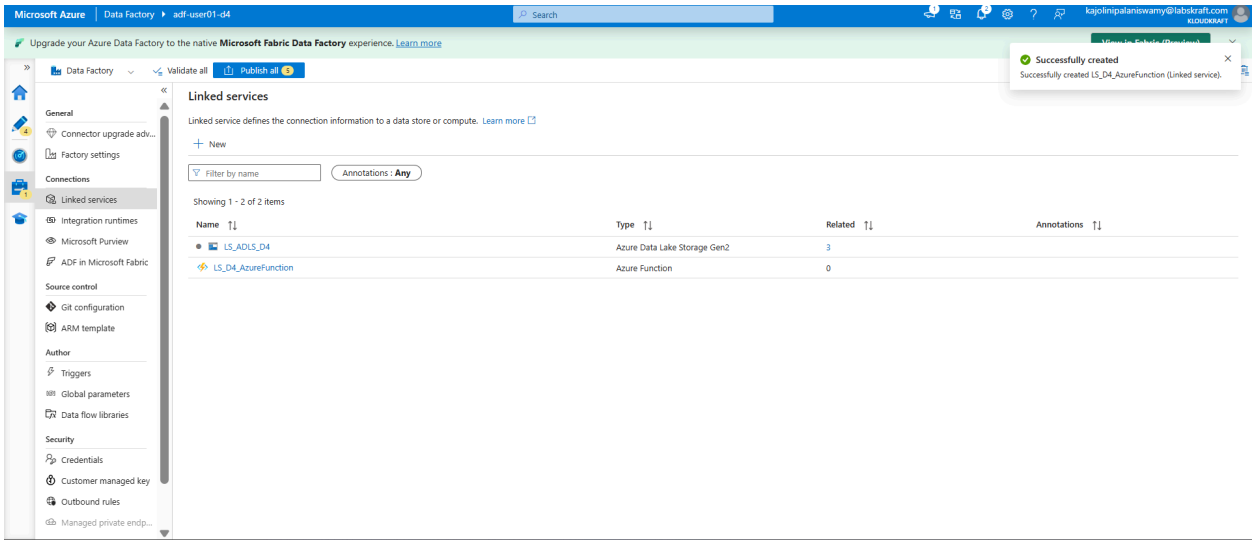
Select Test connection.



Expected result: Connection successful.

Select Create.

ADF uses this linked service to invoke the Function through its Azure Function activity.



Microsoft Learn+1

13. Create the ADF pipeline

Navigate to ADF Studio → Author → + → Pipeline.

Set the pipeline name:

```
None
PL_D4_03_EventToFunction
```

Create a pipeline parameter

1. Select the pipeline canvas.
2. Open the Parameters tab.
3. Add a parameter:

Name	Type
`fileName`	String

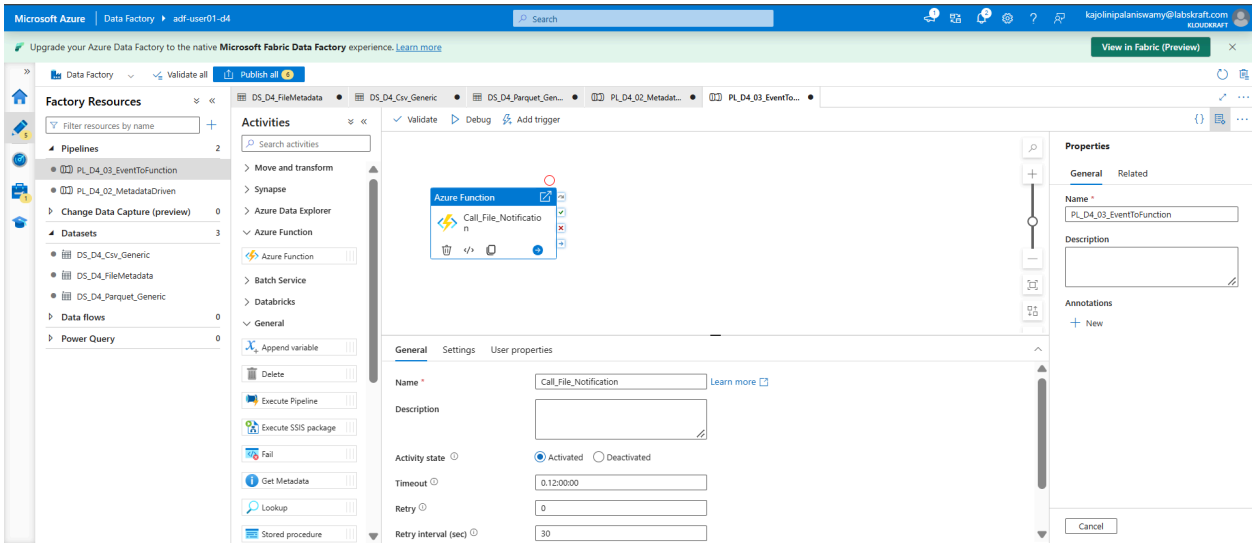
This parameter will receive the uploaded CSV filename from the storage event trigger.

Add Azure Function Activity

1. Open Activities.
2. Expand General.
3. Drag Azure Function onto the canvas.
4. Rename it:

None

```
Call_File_Notification
```



Open the Settings tab.

Configure:

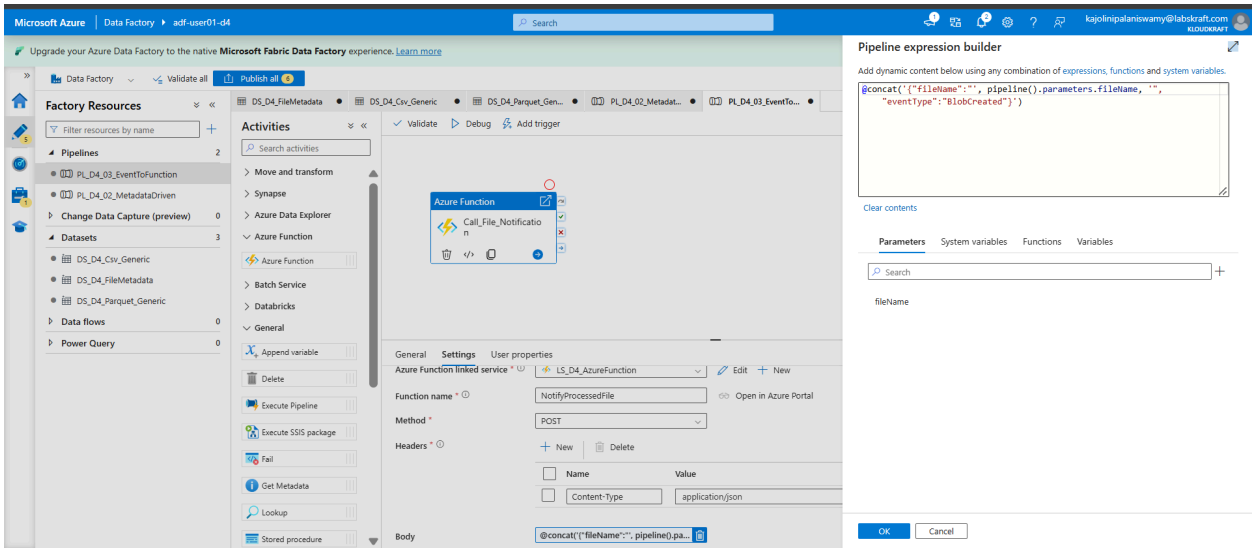
Setting	Value
Azure Function linked service	`LS_D4_AzureFunction`
Function name	`NotifyProcessedFile`
Method	POST

Configure the request body

Under Body, select Add dynamic content.

Use:

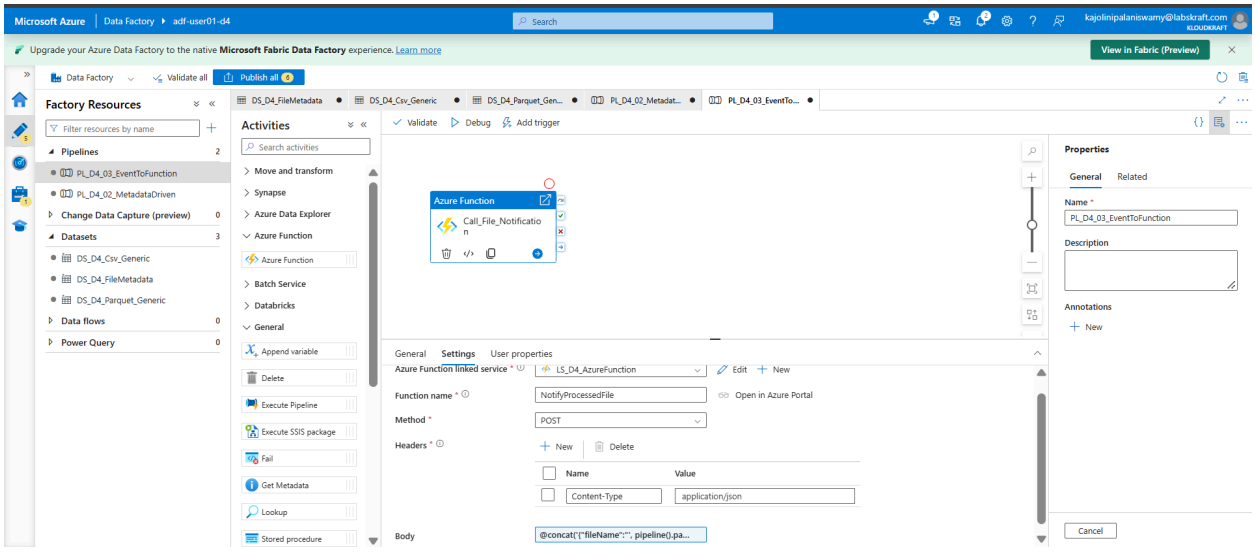
```
None
@concat(
  '{"fileName":"',
  pipeline().parameters.fileName,
  '","eventType":"BlobCreated"}'
)
```



Set the request header:

```
{
  "Content-Type": "application/json"
}
```

The Function receives a JSON notification containing the filename and event type.



Important: The Function must return a JSON object, not a JSON array,

for the ADF Azure Function activity to accept its response. Our C# Function returns an object.

Microsoft Learn+1

14. Create the Storage Event Trigger

Now we configure the automatic pipeline execution.

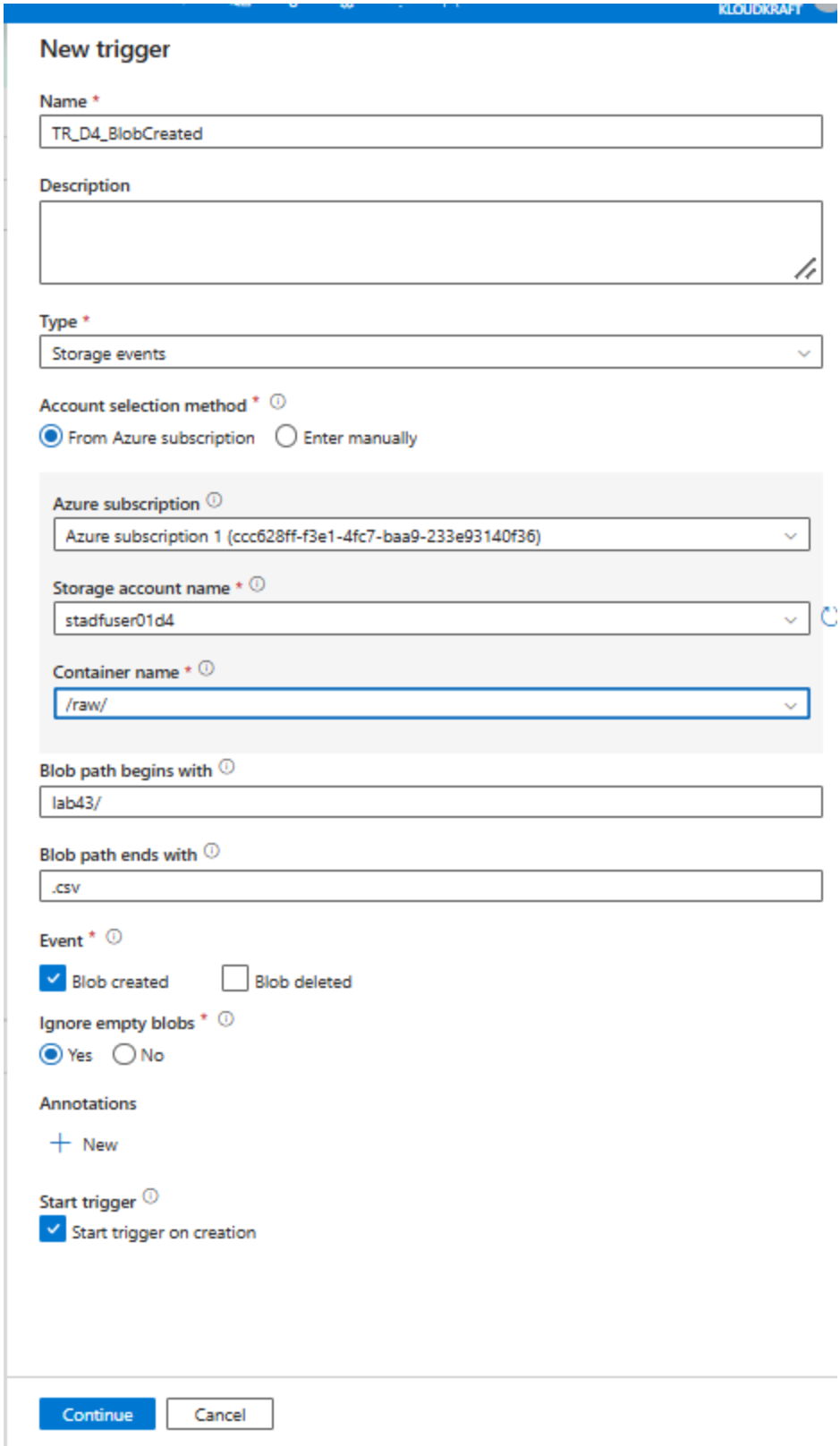
In ADF Studio:

1. Open the pipeline `PL_D4_03_EventToFunction`.
2. Select Add trigger → New/Edit from the pipeline toolbar.
3. Select + New.
4. Choose trigger type Storage events.

Configure:

Setting	Value
Name	`TR_D4_BlobCreated`
Storage account	Your Day 4 Storage Account
Container	`raw`
Event	Blob created

Blob path begins with	`lab43/`
Blob path ends with	`.csv`
Ignore empty blobs	Enabled


KLOUDKRAFT
New trigger
Name *
TR_D4_BlobCreated
Description
Type *
Storage events
Account selection method *
From Azure subscription
Enter manually
Azure subscription
Azure subscription 1 (ccc628ff-f3e1-4fc7-baa9-233e93140f36)
Storage account name *
stadfuser01d4
Container name *
/raw/
Blob path begins with
lab43/
Blob path ends with
.csv
Event *
Blob created
Blob deleted
Ignore empty blobs *
Yes
No
Annotations
New
Start trigger
Start trigger on creation
Continue
Cancel

The container and path filters restrict the trigger to CSV files arriving in the Lab 4.3 folder.

ADF storage event triggers use Azure Event Grid. The path filters support begins-with and ends-with matching, not arbitrary wildcard patterns.

Microsoft Learn+1

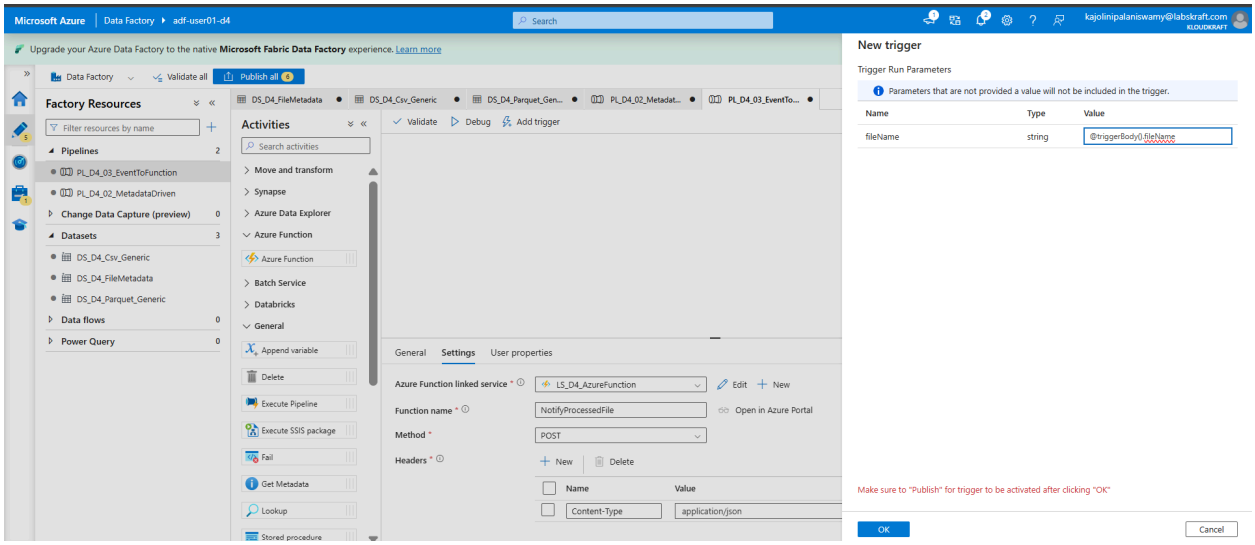
Configure pipeline parameters

In the trigger configuration, map the pipeline parameter:

Pipeline parameter	Dynamic value
`fileName`	`@triggerBody().fileName`

Use the trigger's dynamic-content selector to select the file name field if available, and verify the resulting expression against the first actual event.

This ensures the pipeline receives the name of the uploaded file rather than a hardcoded filename.

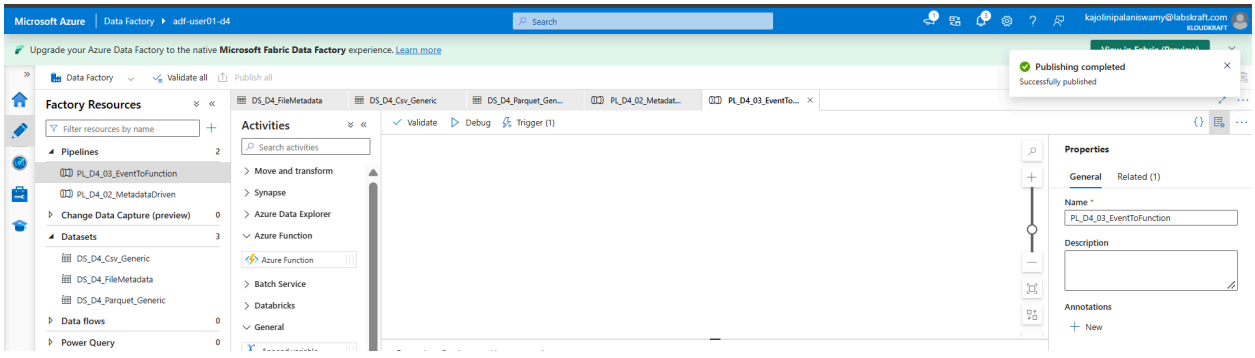


Activate the trigger

1. Review the trigger configuration.
2. Select OK.
3. Select Publish all.
4. Start the trigger if it is not already started.
5. Confirm its status is Started.

Checkpoint 2

The pipeline and storage event trigger are published and active.



15. Upload the test CSV file

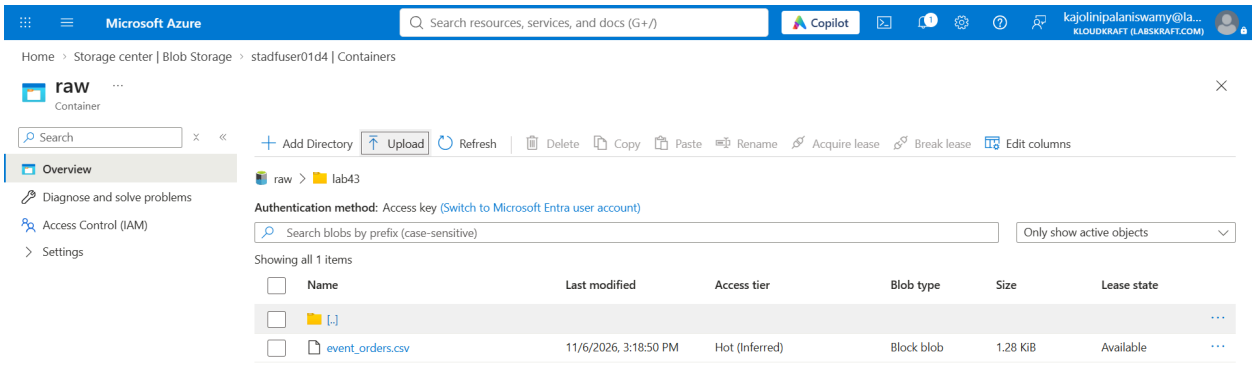
Now we test the complete integration.

Navigate to:

Azure Portal → Storage Account → Containers → raw → lab43

Upload a new CSV file:

```
None
event_orders.csv
```



For the initial test, use the dataset we will include in the final Day 4 artifacts.

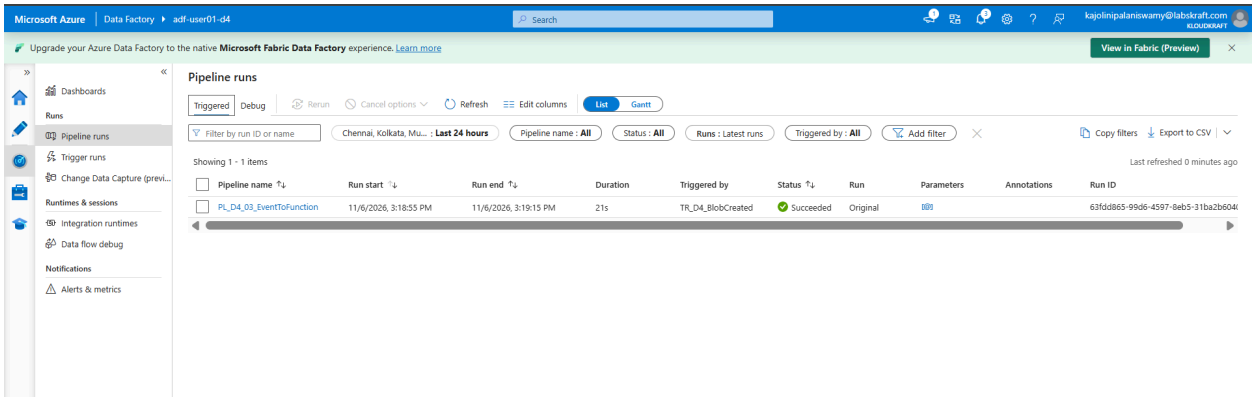
The upload must happen after the trigger is started.

Expected behavior

The following sequence should happen automatically:

1. ADLS Gen2 receives `event_orders.csv`.
2. Storage generates a Blob Created event.
3. Event Grid delivers the event to ADF.

4. ADF starts `PL_D4_03_EventToFunction`.



5. ADF passes the filename to the pipeline parameter.
6. Azure Function Activity calls `NotifyProcessedFile`.
7. The C# Function validates the notification.
8. The Function returns HTTP 200 with an acknowledgement.

16. Validate the ADF pipeline execution

Open ADF Studio → Monitor.

Navigate to Pipeline runs.

Locate:

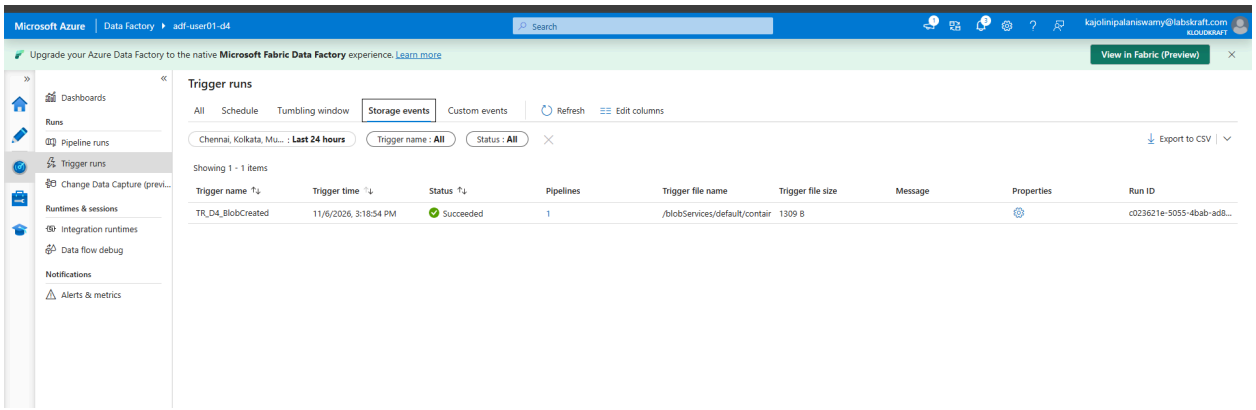
```
None
PL_D4_03_EventToFunction
```

Verify:

- Pipeline run status: Succeeded.
- Trigger name: `TR_D4_BlobCreated`.
- Pipeline parameter `fileName`: `event_orders.csv`.

Open the pipeline run and inspect the activity:

```
None
Call_File_Notification
```



Expected result:

```
None
Activity status: Succeeded
HTTP response: 200
```

Inspect the activity output and confirm the Function returned its acknowledgement.

17. Validate the C# Function execution

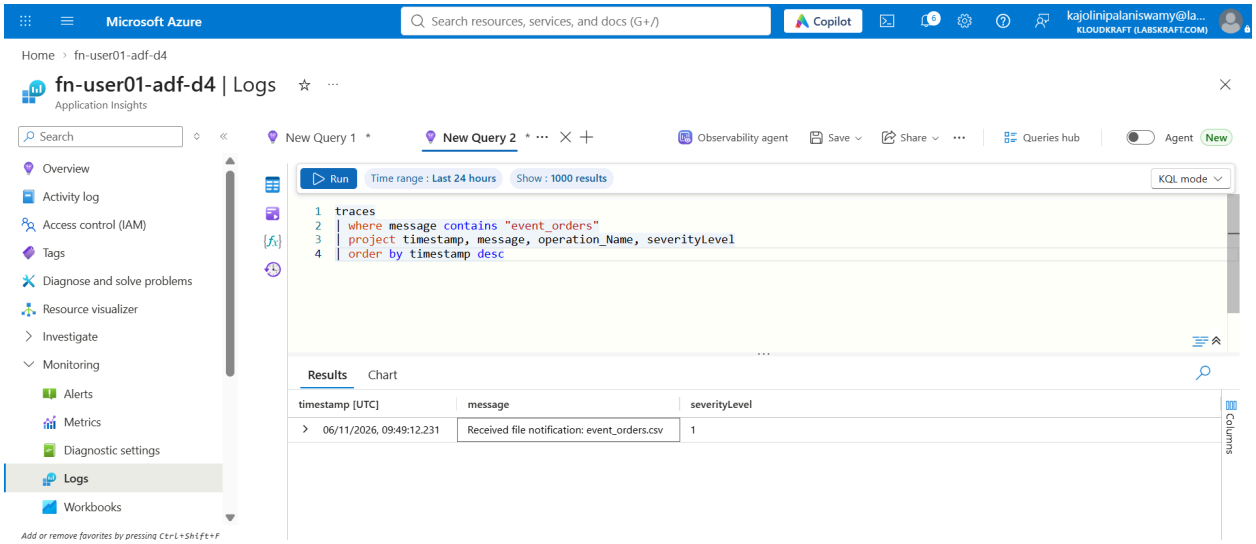
Navigate to:

Azure Portal → Function App → Functions → NotifyProcessedFile

Open Monitor or Application Insights logs.

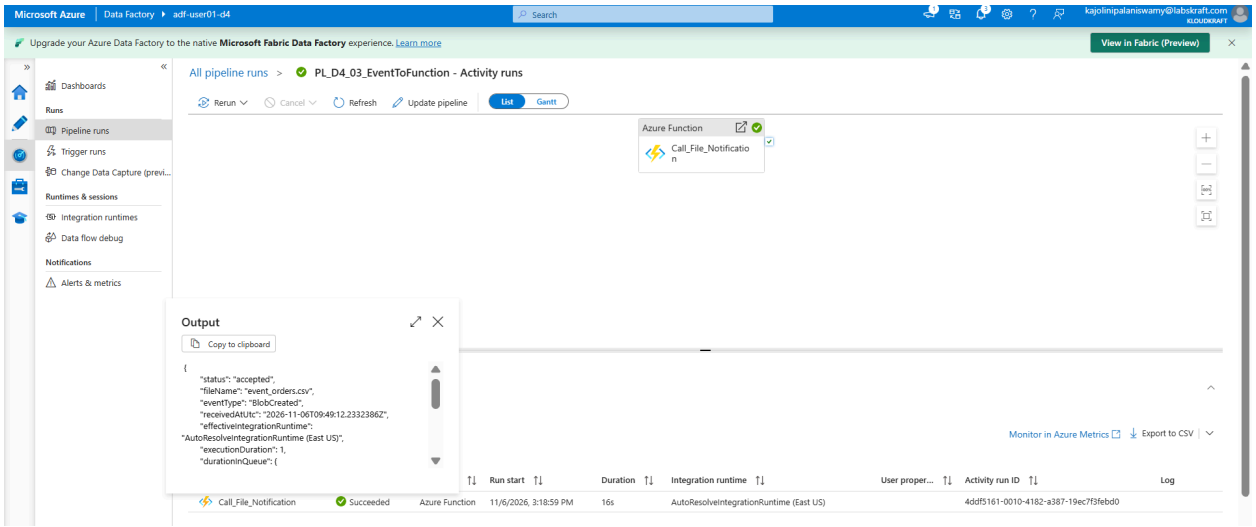
Verify that the log contains:

```
None
Received file notification: event_orders.csv
```



The Function response should contain:

```
None
{
  "status": "accepted",
  "fileName": "event_orders.csv",
  "eventType": "BlobCreated"
}
```



This confirms that the event reached ADF and ADF successfully invoked our C# Function.

Note: This lab validates event-driven orchestration and notification. It does not yet perform CSV transformation or write a Parquet output. Those are the responsibilities of Labs 4.1 and 4.2.

18. Troubleshooting

Problem	What to verify
Trigger does not fire	Trigger status, storage account, container and path filters
Event Grid subscription fails	Event Grid provider registration and subscription permissions
Pipeline does not start	Trigger publication and trigger-run history
Filename is empty	Trigger parameter mapping and actual event payload
Function activity returns 401/403	Function key and linked service
Function activity returns 404	Function name, route and Function App URL

Function activity fails on response	Return a JSON object, not an array
Function logs are missing	Function App health and Application Insights configuration

For repeated tests, upload a new filename each time to generate a new Blob Created event.

19. Evidence collection

Capture these screenshots:

- ADLS Gen2 `raw/lab43` folder containing the test CSV.
- Azure Function App showing `NotifyProcessedFile`.
- Successful local build output.
- ADF Azure Function linked service test.
- ADF pipeline canvas showing the Azure Function Activity.
- Storage Event Trigger configuration.
- Trigger status showing Started.
- ADF Trigger Runs showing the storage event.
- ADF Pipeline Runs showing Succeeded.
- Function Activity output showing HTTP 200.
- C# Function logs showing the received filename.

20. Lab 4.3 completion criteria

Validation	Expected result
C# project	Build successful
Function deployment	Successful
Function endpoint	Available
ADF linked service	Connected
Pipeline configuration	Valid
Event Grid trigger	Started
Blob Created event	Detected
ADF pipeline	Succeeded
Azure Function Activity	Succeeded
Function response	HTTP 200

Function log	Filename recorded